Intelligent Signal Processing Final EOT – Code Submission

Table of Contents

Task 1.1: Vehicle Detection & Tracking 3

#---Install Necessary Packages--- 3

#---Install Necessary Libraries--- 3

#---Details Of Video File & Interested Region Analysis--- 3

~~~Own Implementation – ROI Cropping Logic~~~ 4

#---Vehicle Detection & Tracking Using Background Subtraction--- 5

~~~Own Implementation – Bounding Boxes~~~ 7

Task 1.2: Downtown Car Counter 10

#---Install Necessary Packages--- 10

#---Install Necessary Libraries--- 10

#---Tracking Downtown Moving Cars--- 11

~~~Own Implementation – Centroid Matching Logic~~~ 15

~~~Own Implementation – Position History Management~~~ 16

~~~Own Implementation – Direction Filtering~~~ 17

~~~Own Implementation – Car Removal Logic~~~ 17

Task 2: Audio Compression 20

#---Install Necessary Packages & Libraries--- 20

#---WavPayload - Audio Data Container--- 20

#--- Read WAV File--- 21

#--- Write WAV File--- 22

#--- Zigzag Mapping - Signed to Unsigned Function--- 22

#~~~Own Implementation – Zig Zag Mapping Feature~~~ 22

#---Rice Encode--- 23

#---Rice Decode--- 24

#--- SHA-256 Lossless Verification--- 26
#~~~Own Implementation – SHA-256 Verification~~~ 26
#---Encode WAV -> .ex2 & Decode .ex2 -> WAV--- 26
#---Results Without Delta Prediction--- 27
#---Delta Encoding - Extension--- 28
#~~~Own Implementation – Delta Encoding~~~ 28
#---Encode WAV -> .ex2 & Decode .ex2 -> WAV--- 29
#---Results With Delta Prediction--- 30
#--- Results - Compression Table--- 31
Task 3: Narbonne Film Festival Format Validator 32
#---Install Necessary Libraries--- 32
#---FFMPEG Verification & Installation--- 32
#~~~Own Implementation – Cross Platform Installation Handler~~~ 32
#---Festival Specifications & Functions--- 35
#~~~Own Implementation – Helper Functions~~~ 36
#---Process & Validate Films--- 40
#---Verifying Converted Files--- 43
#~~~Own Implementation – Post Conversion Revalidation Step~~~ 43
#~~~Own Implementation – Detailed Metadata Display~~~ 44

Task 1.1: Vehicle Detection & Tracking

#----------Start of Task 1.1----------

#---Install Necessary Packages---

```
!pip install numpy

!pip install opencv-python
```

#---Install Necessary Libraries---

```
import numpy as np

import cv2

import os
```

#---Details Of Video File & Interested Region Analysis---

```
base_directory = os.getcwd()

print(base_directory)

path_of_vid_file = os.path.join(base_directory, "Exercise1_Files", "Traffic_Laramie_1.mp4")

path_to_save_info = os.path.join(base_directory)


vid_file = cv2.VideoCapture(path_of_vid_file)


# Info of video file

print("~~~ Properties Of Video: ~~~")

print(f"Opened: {vid_file.isOpened()}")

print(f"Width:  {int(vid_file.get(cv2.CAP_PROP_FRAME_WIDTH))}")

print(f"Height: {int(vid_file.get(cv2.CAP_PROP_FRAME_HEIGHT))}")
```

```
print(f"FPS:    {vid_file.get(cv2.CAP_PROP_FPS)}")
print(f"Total frames: {int(vid_file.get(cv2.CAP_PROP_FRAME_COUNT))}")

# To test the interested region we read the first few frames - IR stands for interested region
ret, frame = vid_file.read()
if ret:
    print(f"\n~~~ FRAME 0 SIZE: {frame.shape} (height, width, channels) ~~~")

    # Test interested region coordinates
    # y start - y end to slice height
    IR_TOP, IR_BOTTOM = 265, 590
    # x start - x end to slice width
    IR_LEFT, IR_RIGHT = 0, 1040

    # IR test
    print(f"\n~~~ IR TEST ~~~")
    print(f"IR coords: y = {IR_TOP}:{IR_BOTTOM}, x = {IR_LEFT}:{IR_RIGHT}")
    print(f"IR size:   {(IR_BOTTOM - IR_TOP)} x {(IR_RIGHT - IR_LEFT)}")
    # ~~~Own Implementation – ROI Cropping Logic~~~
    IR = frame[IR_TOP:IR_BOTTOM, IR_LEFT:IR_RIGHT]
    print(f"Actual IR shape: {IR.shape}")

    # Save IR image to inspect visually
    save_image = os.path.join(path_to_save_info, "IR_test.jpg")
    cv2.imwrite(save_image, IR)
    print("IR_test.jpg Saved Successfully!")
```

```
    # Save full frame with IR box
    output = frame.copy()
    cv2.rectangle(output, (IR_LEFT, IR_TOP), (IR_RIGHT, IR_BOTTOM), (0, 0, 255), 3)
    save_frame = os.path.join(path_to_save_info, "frame_with_IR.jpg")
    cv2.imwrite(save_frame, output)
    print("frame_with_IR.jpg Saved Successfully!")
else:
    print("Cannot read first frame. Please check video path!")

vid_file.release()
```

#---Vehicle Detection & Tracking Using Background Subtraction---

```
class TrafficDetector:

    def __init__(self, video_path, min_area=4000):
        # Store path to the video file
        self.video_path    = video_path
        # Minimum contour area which helps to filters out small noise & non-vehicle objects
        self.min_area      = min_area
        self.video_capture = None

        # KNN background subtractor which separates moving cars from static background
        # history: number of past frames used to build background model
        # dist2Threshold: squared distance threshold for pixel classification
        # detectShadows: disabled to avoid shadow pixels being treated as foreground
```

```
        self.background_remover =
cv2.createBackgroundSubtractorKNN(detectShadows=False, history=10000,
dist2Threshold=900)

        self.kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

        # Interested region
        self.IR_TOP, self.IR_BOTTOM = 265, 590
        self.IR_LEFT, self.IR_RIGHT = 0, 1040

    def get_foreground_mask(self, frame):

        # Crop interested region
        ir_frame = frame[self.IR_TOP:self.IR_BOTTOM, self.IR_LEFT:self.IR_RIGHT]

        # Apply Gaussian blur to reduce noise before background subtraction
        smoothed = cv2.GaussianBlur(ir_frame, (5, 5), 0)

        # Apply KNN background subtraction which outputs a mask of cars in white foreground
        mask = self.background_remover.apply(smoothed)

        # Threshold the mask to strict black & white removing any grey shadow pixels
        _, mask = cv2.threshold(mask, 100, 255, cv2.THRESH_BINARY)

        # Morphological closing that fills small holes and gaps within detected car shapes
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, self.kernel, iterations=2)
```

```
    return mask

  def get_bounding_boxes(self, mask):

    # Find external contours of all white regions in the mask
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    # Filter out contours too small to be vehicles
    contours = [c for c in contours if cv2.contourArea(c) > self.min_area]

    bounding_boxes = []
    for contour in contours:
      # Get bounding rectangle for each contour
      x, y, w, h = cv2.boundingRect(contour)

      # Offset coordinates back to full frame position
```

~~~Own Implementation – Bounding Boxes~~~

```
      bounding_boxes.append((
        x + self.IR_LEFT,
        y + self.IR_TOP,
        x + self.IR_LEFT + w,
        y + self.IR_TOP + h
      ))
    return bounding_boxes
```

```
def draw_boxes(self, frame, bounding_boxes):

    # Draw a green rectangle around each detected vehicle
    for (x, y, x2, y2) in bounding_boxes:
        cv2.rectangle(frame, (x, y), (x2, y2), (0, 255, 0), 2)

    # Draw red rectangle showing the monitored IR boundary
    cv2.rectangle(frame,
                  (self.IR_LEFT, self.IR_TOP),
                  (self.IR_RIGHT, self.IR_BOTTOM),
                  (0, 0, 255), 2)

def open_video(self):

    # Load the video file from the given path
    self.video_capture = cv2.VideoCapture(self.video_path)

    # Check if the video opened successfully else print error and return False if not
    if not self.video_capture.isOpened():
        print(f"Error: Could not open video: {self.video_path}")
        return False
    return True

def run(self):

    # Open video and exit if it fails
```

```
if not self.open_video():
    return

# Get the video frame rate to sync playback speed
fps = self.video_capture.get(cv2.CAP_PROP_FPS)

# Calculate wait time per frame in milliseconds
wait_ms = int(1000 / (fps * 3))

while True:
    # Read the next frame from the video
    ret, frame = self.video_capture.read()

    # Stop loop if no more frames are available
    if not ret:
        break

    # Generate the foreground mask from the current frame
    mask = self.get_foreground_mask(frame)

    # Get bounding boxes of detected vehicles
    bounding_boxes = self.get_bounding_boxes(mask)

    # Draw boxes and IR boundary on the frame
    self.draw_boxes(frame, bounding_boxes)
```

```
        # Display the annotated frame in the Live Video window
        cv2.imshow("Live Video", frame)

        # Exit the loop if the user presses 'q'
        if cv2.waitKey(wait_ms) & 0xFF == ord('q'):
            break

    # Release video resources and close all display windows
    self.video_capture.release()
    cv2.destroyAllWindows()



# Run code section
path_of_vid_file = os.path.join(base_directory, "Exercise1_Files", "Traffic_Laramie_1.mp4")
detector = TrafficDetector(video_path=path_of_vid_file, min_area=4000)
detector.run()
```

#----------End of Task 1.1----------

Task 1.2: Downtown Car Counter

#----------Start of Task 1.2----------

#---Install Necessary Packages---

```
!pip install numpy
!pip install opencv-python
```

#---Install Necessary Libraries---

```
import numpy as np
```

```
import cv2
import os
base_directory = os.getcwd()
print(base_directory)
```

#---Tracking Downtown Moving Cars---

```
class Car:

  def __init__(self, id, position):
    # Unique ID for this car
    self.id = id
    # Current position of the car's center
    self.position = position
    # Previous position used to compute movement direction
    self.prev_position = position
    # Flag to ensure each car is counted only once
    self.counted = False
    # Number of consecutive frames the car was not detected
    self.frames_missing = 0
    # Number of frames this car has been actively tracked
    self.frames_tracked = 0
    # Last 5 positions used to compute average movement direction
    self.position_history = [position]


def is_inside_zone(center, roi_polygon):
```

```
    return cv2.pointPolygonTest(roi_polygon, (float(center[0]), float(center[1])), False) >= 0



def distance(p1, p2):
    return np.sqrt((p1[0] - p2[0])**2 + (p1[1] - p2[1])**2)



def analyseVideo(video_path):

    # Open video file
    video = cv2.VideoCapture(video_path)
    if not video.isOpened():
        print(f"Error: Could not open video: {video_path}")
        return 0, 0

    # Get video properties
    fps        = video.get(cv2.CAP_PROP_FPS)
    video_width  = int(video.get(cv2.CAP_PROP_FRAME_WIDTH))
    video_height = int(video.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(video.get(cv2.CAP_PROP_FRAME_COUNT))
    duration_sec = total_frames / fps

    print(f"Video: {video_path.split(os.sep)[-1]}")
    print(f"Size : {video_width}x{video_height} | FPS:{fps:.1f} | Duration:{duration_sec:.1f}s")

    # KNN background subtractor which separates moving cars from static background
```

```
# history: number of past frames to build background model
# dist2Threshold: sensitivity of foreground detection
# detectShadows: disabled to avoid shadow pixels being treated as cars
background_remover = cv2.createBackgroundSubtractorKNN(
    detectShadows=False,
    history=500,
    dist2Threshold=400
)

# Kernel for morphological closing to fill gaps in detected shapes
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5, 5))

# Downtown counting zone whereby only cars passing through this box are counted
LEFT, TOP, RIGHT, BOTTOM = 840, 200, 1010, 345
center_x   = (LEFT + RIGHT)  / 2.0
center_y   = (TOP  + BOTTOM) / 2.0
roi_width  = float(RIGHT - LEFT)
roi_height = float(BOTTOM - TOP)

# Counters and trackers
vehicle_count = 0
car_id        = 0
frame_count   = 0
cars          = {}

while video.isOpened():
```

```
    ret, frame = video.read()
    if not ret:
        break

    frame_count += 1

    # Increment missing frame counter for all currently tracked cars
    for car in cars.values():
        car.frames_missing += 1

    # Apply Gaussian blur to reduce noise before background subtraction
    smoothed_frame  = cv2.GaussianBlur(frame, (5, 5), 0)

    # Apply KNN background subtraction to get foreground mask
    foreground_mask = background_remover.apply(smoothed_frame)

    # Convert mask to strict black & white — removes grey shadow pixels
    _, foreground_mask = cv2.threshold(foreground_mask, 100, 255, cv2.THRESH_BINARY)

    # Morphological closing that fills small holes and gaps in detected car shapes
    foreground_mask = cv2.morphologyEx(foreground_mask, cv2.MORPH_CLOSE, kernel,
iterations=2)

    # Find external contours of all white regions in the mask
    contours, _ = cv2.findContours(foreground_mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
```

```
# Build the downtown counting zone polygon
rect       = ((center_x, center_y), (roi_width, roi_height), 0)
roi_polygon = cv2.boxPoints(rect).astype(np.int32)

detected_in_zone = 0

for contour in contours:
    x, y, w, h = cv2.boundingRect(contour)
    area       = cv2.contourArea(contour)

    # Filter out contours too small or too large to be vehicles
    if area < 3000 or area > 50000:
        continue

    # Compute the center of this detection
    center = (x + w // 2, y + h // 2)

    # Find closest existing car within matching distance
    matched_car = None
    min_dist    = 200
```

~~~Own Implementation – Centroid Matching Logic~~~

```
    for car in cars.values():
        dist = distance(center, car.position)
        if dist < min_dist:
            min_dist    = dist
            matched_car = car
```

~~~Own Implementation – Position History Management~~~

```
if matched_car is not None:
    # Update existing car's position and tracking history
    matched_car.prev_position  = matched_car.position
    matched_car.position       = center
    matched_car.frames_missing = 0
    matched_car.frames_tracked += 1
    matched_car.position_history.append(center)
    # Keep only last 5 positions for direction calculation
    if len(matched_car.position_history) > 5:
        matched_car.position_history.pop(0)
else:
    # If no match found then register as a new car
    cars[car_id] = Car(car_id, center)
    matched_car  = cars[car_id]
    car_id     += 1

# Check if car is inside the downtown counting zone
if is_inside_zone(center, roi_polygon):
    detected_in_zone += 1

    if not matched_car.counted:
        # Need at least 3 positions to compute a reliable direction
        if len(matched_car.position_history) >= 3:
            # Compute average horizontal displacement across tracked positions
```

~~~Own Implementation – Direction Filtering~~~

```
            avg_dx = (
                matched_car.position_history[-1][0] -
                matched_car.position_history[0][0]
            ) / len(matched_car.position_history)


            # Downtown cars if avg_dx < 1 else avg_dx >= 1 are blocked
            if avg_dx < 1:
                vehicle_count     += 1
                matched_car.counted = True
                print(f"Car #{vehicle_count} to downtown | ID:{matched_car.id} |
Frame:{frame_count} | Pos:{center} | avg_dx:{avg_dx:.2f}")


        # Green box for cars inside the zone
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 3)
        cv2.putText(frame, f"{int(area)}", (x, y - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
    else:
        # Blue box for cars outside the zone
        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

~~~Own Implementation – Car Removal Logic~~~

```
    # Remove cars that have been missing for more than 10 frames
    for cid in [cid for cid, c in cars.items() if c.frames_missing > 10]:
        del cars[cid]
```

```
    # Draw the downtown counting zone boundary in red
    cv2.drawContours(frame, [roi_polygon], 0, (0, 0, 255), 4)
    cv2.putText(frame, "TO DOWNTOWN",
            (int(center_x) - 70, int(center_y) - int(roi_height / 2) - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

    # Live display running stats on the frame
    elapsed_min = (frame_count / fps) / 60
    cpm       = vehicle_count / elapsed_min if elapsed_min > 0 else 0
    cv2.putText(frame, f"Downtown Cars: {vehicle_count}",
            (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 2)
    cv2.putText(frame, f"Cars/min: {cpm:.2f}",
            (10, 65), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)
    cv2.putText(frame, f"Frame: {frame_count} | In zone: {detected_in_zone}",
            (10, 95), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 2)

    cv2.imshow("Downtown Counter", frame)
    if cv2.waitKey(10) == ord("q"):
        break

video.release()
cv2.destroyAllWindows()

# Final results
cpm_final = vehicle_count / (duration_sec / 60) if duration_sec > 0 else 0
print("\n" + "=" * 50)
```

```
    print(f"TOTAL CARS TO DOWNTOWN: {vehicle_count}")
    print(f"CARS PER MINUTE      : {cpm_final:.2f}")
    print("=" * 50)
    return vehicle_count, cpm_final



# Run code section
video1 = os.path.join(base_directory, "Exercise1_Files", "Traffic_Laramie_1.mp4")
video2 = os.path.join(base_directory, "Exercise1_Files", "Traffic_Laramie_2.mp4")

print("### Traffic_Laramie_1.mp4 ###")
total1, cpm1 = analyseVideo(video1)

print("\n### Traffic_Laramie_2.mp4 ###")
total2, cpm2 = analyseVideo(video2)

print("\n" + "=" * 60)
print("TASK 1.2 RESULTS:")
print("=" * 60)
print(f"| {'Video':<23} | {'Total Cars':>10} | {'Cars/Minute':>11} |")
print(f"|-{'':-<23}-|-{'':-<10}-|-{'':-<11}-|")
print(f"| {'Traffic_Laramie_1.mp4':<23} | {total1:>10} | {cpm1:>11.2f} |")
print(f"| {'Traffic_Laramie_2.mp4':<23} | {total2:>10} | {cpm2:>11.2f} |")
print("=" * 60)
```

----------End of Task 1.2----------

Task 2: Audio Compression

#----------Start of Task 2----------

#---Install Necessary Packages & Libraries---

```
%pip install tabulate pandas numpy


# Import Library

import csv

# For SHA-256 lossless verification

import hashlib

import struct

import wave

import os

import numpy as np

import pandas as pd

from dataclasses import dataclass

from pathlib import Path

from typing import List, Sequence


# Magic bytes used to identify our custom .ex2 file format

MAGIC = b"RCEX2"
```

#---WavPayload - Audio Data Container---

```
@dataclass

class WavPayload:

    sample_rate: int
```

```
    channels: int
    sample_width: int
    samples: np.ndarray
```

#--- Read WAV File---

```
def read_wav_mono(path: Path) -> WavPayload:
    with wave.open(str(path), "rb") as wf:
        channels = wf.getnchannels()
        sample_width = wf.getsampwidth()
        sample_rate = wf.getframerate()
        n_frames = wf.getnframes()
        raw = wf.readframes(n_frames)

    # Only 16-bit PCM is supported
    if sample_width != 2:
        raise ValueError("Only 16-bit PCM WAV is supported.")

    # Convert raw bytes to signed 16-bit integer array
    samples = np.frombuffer(raw, dtype=np.int16)

    # If stereo, reshape to (n_frames, 2) and keep only first channel
    if channels > 1:
        samples = samples.reshape(-1, channels)[:, 0]

    return WavPayload(
        sample_rate=sample_rate,
```

```
        channels=1,
        sample_width=sample_width,
        samples=samples.copy(),
    )
```

#--- Write WAV File---

```
def write_wav_mono(path: Path, payload: WavPayload) -> None:
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)

    with wave.open(str(path), "wb") as wf:
        wf.setnchannels(1)
        wf.setsampwidth(payload.sample_width)
        wf.setframerate(payload.sample_rate)
        wf.writeframes(payload.samples.astype(np.int16).tobytes())
```

#--- Zigzag Mapping - Signed to Unsigned Function---

#~~~Own Implementation – Zig Zag Mapping Feature~~~

```
def signed_to_unsigned(arr: np.ndarray) -> np.ndarray:
    a = arr.astype(np.int32)
    return np.where(a >= 0, a << 1, ((-a) << 1) - 1).astype(np.uint32)

def unsigned_to_signed(u: np.ndarray) -> np.ndarray:
    u = u.astype(np.uint32)
    return np.where(u % 2 == 0, u // 2, -((u + 1) // 2)).astype(np.int32)
```

#---Rice Encode---

```
def rice_encode(samples: np.ndarray, k: int) -> bytes:
    arr = samples.astype(np.int32)
    u   = signed_to_unsigned(arr)

    q = (u >> k).astype(np.uint32)
    r = (u & ((1 << k) - 1)).astype(np.uint32)

    # Codeword start positions in the bitstream
    codeword_lengths = (q + 1 + k).astype(np.int64)
    starts           = np.zeros(len(u) + 1, dtype=np.int64)
    np.cumsum(codeword_lengths, out=starts[1:])
    total_bits = int(starts[-1])

    # Ffill unary ones using cumsum marker trick
    # Set +1 at codeword start, -1 at delimiter position -> cumsum gives 1s in between
    delim_pos = (starts[:-1] + q).astype(np.int64)
    marker    = np.zeros(total_bits + 1, dtype=np.int32)
    np.add.at(marker, starts[:-1], 1)
    np.add.at(marker, delim_pos,  -1)
    bitstream        = (np.cumsum(marker[:-1]) > 0).astype(np.uint8)
    bitstream[delim_pos] = 0  # delimiter zeros

    # Place remainder bits using vectorised index array
```

```
    if k > 0:
        rem_starts  = delim_pos + 1
        rem_indices = (rem_starts[:, None] + np.arange(k)).astype(np.int64)
        shifts      = np.arange(k - 1, -1, -1, dtype=np.uint32)
        rem_bits    = ((r[:, None] >> shifts) & 1).astype(np.uint8)
        bitstream[rem_indices] = rem_bits

    # Pack bits to bytes using np.packbits
    pad    = (8 - total_bits % 8) % 8
    padded = np.concatenate([bitstream, np.zeros(pad, dtype=np.uint8)])
    packed = np.packbits(padded)

    return struct.pack("<I", total_bits) + packed.tobytes()
```

#---Rice Decode---

```
def rice_decode(encoded: bytes, k: int, expected_count: int) -> np.ndarray:
    total_bits = struct.unpack("<I", encoded[:4])[0]
    raw        = np.frombuffer(encoded[4:], dtype=np.uint8)

    # Unpack all bits at once
    bits     = np.unpackbits(raw)[:total_bits]
    zero_pos = np.where(bits == 0)[0]  # all delimiter positions

    codeword_starts = np.empty(expected_count, dtype=np.int64)
    delimiters      = np.empty(expected_count, dtype=np.int64)
```

```
# Walk codewords using searchsorted
pos = 0
for i in range(expected_count):
    idx = int(np.searchsorted(zero_pos, pos))
    if idx >= len(zero_pos):
        break
    delimiters[i]     = zero_pos[idx]
    codeword_starts[i] = pos
    pos               = int(zero_pos[idx]) + k + 1

q = (delimiters - codeword_starts).astype(np.uint32)

if k > 0:
    rem_starts  = delimiters + 1
    rem_indices = (rem_starts[:, None] + np.arange(k)).astype(np.int64)
    shifts      = np.arange(k - 1, -1, -1, dtype=np.uint32)
    rem_bits    = bits[rem_indices]
    r           = (rem_bits * (1 << shifts)).sum(axis=1).astype(np.uint32)
else:
    r = np.zeros(expected_count, dtype=np.uint32)

u = ((q << k) | r).astype(np.uint32)
return unsigned_to_signed(u).astype(np.int16)
```

#--- SHA-256 Lossless Verification---

#~~~Own Implementation – SHA-256 Verification~~~

```
def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        while True:
            chunk = f.read(1024 * 1024)
            if not chunk:
                break
            h.update(chunk)
    return h.hexdigest()
```

#---Encode WAV -> .ex2 & Decode .ex2 -> WAV---

```
def encode_wav_to_ex2(wav_path: Path, ex2_path: Path, k: int) -> None:
    payload = read_wav_mono(wav_path)
    body    = rice_encode(payload.samples, k)  # raw samples — no delta
    header  = struct.pack(
        "<5sIHHIB", MAGIC,
        payload.sample_rate, payload.channels,
        payload.sample_width, len(payload.samples), k
    )
    Path(ex2_path).parent.mkdir(parents=True, exist_ok=True)
    Path(ex2_path).write_bytes(header + body)



def decode_ex2_to_wav(ex2_path: Path, wav_out_path: Path) -> None:
```

```
    blob  = Path(ex2_path).read_bytes()
    hfmt  = "<5sIHHIB"
    hlen  = struct.calcsize(hfmt)
    magic, sr, ch, sw, n, k = struct.unpack(hfmt, blob[:hlen])
    if magic != MAGIC:
        raise ValueError("Not a valid EX2 file")
    samples = rice_decode(blob[hlen:], k, n)
    write_wav_mono(wav_out_path, WavPayload(sr, ch, sw, samples))
```

#---Results Without Delta Prediction---

```
base_dir   = Path(".").resolve()
input_dir  = base_dir / "Exercise2_Files"
output_dir = base_dir / "outputs_no_delta"
output_dir.mkdir(parents=True, exist_ok=True)

rows_no_delta = []

for wav_name in ["Sound1.wav", "Sound2.wav"]:
    src          = input_dir / wav_name
    original_size = src.stat().st_size
    row          = {"File": wav_name, "Original Size": original_size}

    for k in [4, 2]:
        enc = output_dir / f"{src.stem}_NoDelta_K{k}.ex2"
        dec = output_dir / f"{src.stem}_NoDelta_K{k}_dec.wav"
```

```
        print(f"Encoding {wav_name} K={k}...\n", end=" ", flush=True)
        encode_wav_to_ex2(src, enc, k)

        decode_ex2_to_wav(enc, dec)

        ok  = sha256(src) == sha256(dec)
        cs  = enc.stat().st_size
        pct = 100.0 * (1.0 - cs / original_size)

        row[f"Rice K={k} Size"] = cs
        row[f"% Compression K={k}"] = f"{pct:.2f}%"

    rows_no_delta.append(row)

df_no_delta = pd.DataFrame(rows_no_delta)
print("\nTable 2.1 - Without Delta Prediction")
display(df_no_delta)
```

#---Delta Encoding - Extension---

#~~~Own Implementation – Delta Encoding~~~

```
def delta_encode(samples: np.ndarray) -> np.ndarray:
    s      = samples.astype(np.int32)
    out    = np.empty_like(s)
    out[0]  = int(s[0])
    diff    = s[1:] - s[:-1]
```

```
    out[1:] = ((diff + 32768) % 65536) - 32768
    return out



def delta_decode(delta: np.ndarray) -> np.ndarray:
    values  = np.array(delta, dtype=np.int32)
    decoded = np.cumsum(values).astype(np.int32)
    decoded = ((decoded + 32768) % 65536) - 32768
    return decoded.astype(np.int16)
```

#---Encode WAV -> .ex2 & Decode .ex2 -> WAV---

```
def encode_wav_to_ex2_delta(wav_path: Path, ex2_path: Path, k: int) -> None:
    payload   = read_wav_mono(wav_path)
    residuals = delta_encode(payload.samples)   # Extension: delta first
    body     = rice_encode(residuals, k)
    header    = struct.pack(
        "<5sIHHIB", MAGIC,
        payload.sample_rate, payload.channels,
        payload.sample_width, len(payload.samples), k
    )
    Path(ex2_path).parent.mkdir(parents=True, exist_ok=True)
    Path(ex2_path).write_bytes(header + body)



def decode_ex2_to_wav_delta(ex2_path: Path, wav_out_path: Path) -> None:
    blob  = Path(ex2_path).read_bytes()
    hfmt  = "<5sIHHIB"
```

```
    hlen  = struct.calcsize(hfmt)
    magic, sr, ch, sw, n, k = struct.unpack(hfmt, blob[:hlen])
    if magic != MAGIC:
        raise ValueError("Not a valid EX2 file")
    residuals = rice_decode(blob[hlen:], k, n)
    samples   = delta_decode(residuals)
    write_wav_mono(wav_out_path, WavPayload(sr, ch, sw, samples))
```

#---Results With Delta Prediction---

```
output_dir_delta = base_dir / "outputs_with_delta"
output_dir_delta.mkdir(parents=True, exist_ok=True)

rows_delta = []

for wav_name in ["Sound1.wav", "Sound2.wav"]:
    src         = input_dir / wav_name
    original_size = src.stat().st_size
    row         = {"File": wav_name, "Original Size": original_size}

    for k in [4, 2]:
        enc = output_dir_delta / f"{src.stem}_Delta_K{k}.ex2"
        dec = output_dir_delta / f"{src.stem}_Delta_K{k}_dec.wav"

        print(f"Encoding {wav_name} K={k}...\n", end=" ", flush=True)
        encode_wav_to_ex2_delta(src, enc, k)
```

```
        decode_ex2_to_wav_delta(enc, dec)

        ok  = sha256(src) == sha256(dec)
        cs  = enc.stat().st_size
        pct = 100.0 * (1.0 - cs / original_size)

        row[f"Rice K={k} Size"] = cs
        row[f"% Compression K={k}"] = f"{pct:.2f}%"

    rows_delta.append(row)

df_delta = pd.DataFrame(rows_delta)
print("\nTable 2.2 — With Delta Prediction (Extension)")
display(df_delta)
```

#--- Results - Compression Table---

```
comparison = []
files = ["Sound1.wav", "Sound2.wav"]
for i, wav_name in enumerate(files):
    for k in [4, 2]:
        no_delta_pct = rows_no_delta[i][f"% Compression K={k}"]
        delta_pct    = rows_delta[i][f"% Compression K={k}"]
        comparison.append({
            "File"           : wav_name,
            "K"              : k,
            "Without Delta (%)" : no_delta_pct,
```

```
        "With Delta (%)"    : delta_pct,
    })

df_comparison = pd.DataFrame(comparison)
print("\nTable 2.3 — Without vs With Delta Prediction")
display(df_comparison)
```

----------End of Task 2----------

Task 3: Narbonne Film Festival Format Validator

#----------Start of Task 3----------

#---Install Necessary Libraries---

```
import json
import subprocess
from pathlib import Path
from math import gcd
import glob
import subprocess
import platform
```

#---FFMPEG Verification & Installation---

#~~~Own Implementation – Cross Platform Installation Handler~~~

```
def check_and_install_ffmpeg():

  # Check if ffmpeg is already installed
  try:
    result = subprocess.run(
```

```
        ["ffmpeg", "-version"],
        capture_output=True,
        text=True
    )
    if result.returncode == 0:
        print("ffmpeg already installed")
        return
except FileNotFoundError:
    print("ffmpeg not found. Attempting to install!\n")

os_type = platform.system()

if os_type == "Darwin":
    # macOS
    try:
        print("macOS detected. Installing via Homebrew!")
        subprocess.run(["brew", "install", "ffmpeg"], check=True)
        print("ffmpeg installed via Homebrew")
    except Exception as e:
        print(f"Homebrew installation failed: {e}")

elif os_type == "Linux":
    # Linux
    try:
        print("Linux detected. Installing via apt-get!")
        subprocess.run(["apt-get", "install", "-y", "ffmpeg"], check=True)
```

```
        print("ffmpeg installed via apt-get!")
    except Exception as e:
        print(f"apt-get installation failed: {e}")

else:
    # Windows
    print("Windows detected. Manual installation required!")
    print("Download from official site:")
    print("https://ffmpeg.org/download.html")
    print("Use This Link As Guide If Not Sure With Installation: https://www.youtube.com/watch?v=KBnyOH1o5Ms")

check_and_install_ffmpeg()

# Verify both tools are accessible before running pipeline
try:
    result = subprocess.run(["ffprobe", "-version"], capture_output=True, text=True)
    ffprobe_bin = "ffprobe" if result.returncode == 0 else None
except FileNotFoundError:
    ffprobe_bin = None

try:
    result = subprocess.run(["ffmpeg", "-version"], capture_output=True, text=True)
    ffmpeg_bin = "ffmpeg" if result.returncode == 0 else None
except FileNotFoundError:
    ffmpeg_bin = None
```

```
if not ffprobe_bin or not ffmpeg_bin:
    raise EnvironmentError("ffmpeg/ffprobe not available. Install before continuing!")

print(f"ffprobe: {ffprobe_bin} | ffmpeg: {ffmpeg_bin}")
```

#---Festival Specifications & Functions---

```
# Festival Specifications
target_specs_required = {
    "container":       "mp4",
    # h.265
    "video_codec":      "hevc",
    "audio_codec":      "aac",
    "fps":             25.0,
    "aspect":          "16:9",
    "resolution":       (640, 360),
    # 2 Mb/s
    "video_bitrate_min": 2_000_000,
    # 5 Mb/s
    "video_bitrate_max": 5_000_000,
    # 256 kb/s
    "audio_bitrate_max": 256_000,
    "audio_channels":    2,
}

print("FESTIVAL SPECIFICATIONS:")
```

```
for key, value in target_specs_required.items():
    print(f"  {key:<22}: {value}")


# Helper Functions
```

#~~~Own Implementation – Helper Functions~~~

```
def parse_fps(ratio):
    if not ratio or ratio == "0/0":
        return 0.0
    num, den = ratio.split("/")
    return float(num) / float(den)


def calc_aspect_ratio(width, height):
    common = gcd(width, height)
    return f"{width // common}:{height // common}"


def bitrate_to_mbps(bitrate):
    return f"{bitrate / 1_000_000:.2f} Mb/s"


def bitrate_to_kbps(bitrate):
    return f"{bitrate / 1_000:.1f} kb/s"


# ffprobe wrapper
def probe_file(filepath):
    cmd = [
        ffprobe_bin,
        "-v",
```

```
        "error",
        "-show_entries",
        "format=format_name,bit_rate:"
        "stream=index,codec_type,codec_name,width,height,"
        "r_frame_rate,bit_rate,channels",
        "-of",
        "json",
        str(filepath),
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    if result.returncode != 0:
        raise RuntimeError(f"ffprobe failed: {result.stderr}")
    return json.loads(result.stdout)

# Validation
def validate_file(filepath):
    issues   = []
    data     = probe_file(filepath)
    v_stream = next((s for s in data["streams"] if s.get("codec_type") == "video"), None)
    a_stream = next((s for s in data["streams"] if s.get("codec_type") == "audio"), None)

    # 1.Container
    format_name = str(data.get("format", {}).get("format_name", ""))
    if target_specs_required["container"] not in format_name:
        issues.append(f"Container: expected mp4, got '{format_name}'")
```

```
if v_stream:
    # 2.Video codec
    v_codec = v_stream.get("codec_name", "")
    if v_codec != target_specs_required["video_codec"]:
        issues.append(f"Video codec: expected hevc, got '{v_codec}'")

    # 3.Resolution
    w = int(v_stream.get("width")  or 0)
    h = int(v_stream.get("height") or 0)
    if (w, h) != target_specs_required["resolution"]:
        issues.append(f"Resolution: expected 640x360, got {w}x{h}")

    # 4.Fps
    fps = parse_fps(str(v_stream.get("r_frame_rate", "0/0")))
    if abs(fps - target_specs_required["fps"]) > 0.15:
        issues.append(f"FPS: expected ~25, got {fps:.2f}")

    # 5.Aspect ratio
    if h > 0:
        aspect = calc_aspect_ratio(w, h)
        if aspect != target_specs_required["aspect"]:
            issues.append(f"Aspect ratio: expected 16:9, got {aspect}")

    # 6.Video bitrate
    vbr = int(v_stream.get("bit_rate") or 0)
```

```
        if not (target_specs_required["video_bitrate_min"] <= vbr <=
target_specs_required["video_bitrate_max"]):
            issues.append(f"Video bitrate: expected 2-5 Mb/s, got {bitrate_to_mbps(vbr)}")

    if a_stream:
        # 7.Audio codec
        a_codec = a_stream.get("codec_name", "")
        if a_codec != target_specs_required["audio_codec"]:
            issues.append(f"Audio codec: expected aac, got '{a_codec}'")

        # 8.Audio channels
        channels = int(a_stream.get("channels") or 0)
        if channels != target_specs_required["audio_channels"]:
            issues.append(f"Audio channels: expected 2, got {channels}")

        # 9.Audio bitrate
        abr = int(a_stream.get("bit_rate") or 0)
        if abr > target_specs_required["audio_bitrate_max"]:
            issues.append(f"Audio bitrate: expected ≤256 kb/s, got {bitrate_to_kbps(abr)}")

    return len(issues) == 0, issues

# Conversion
def convert_file(input_path, output_path):
    Path(output_path).parent.mkdir(parents=True, exist_ok=True)
    cmd = [
```

```
        ffmpeg_bin, "-y", "-i", str(input_path),
        "-vf",
        "scale=640:360,fps=25,setsar=1",
        "-c:v",
        "libx265",
        "-b:v",
        "3M",
        "-maxrate",
        "5M",
        "-bufsize",
        "6M",
        "-pix_fmt",
        "yuv420p",
        "-c:a",
        "aac",
        "-b:a",
        "192k",
        "-ac",
        "2",
        str(output_path),
    ]
    result = subprocess.run(cmd, capture_output=True, text=True)
    return result.returncode == 0
```

#---Process & Validate Films---

Locate all input video files

```
video_files = (
    glob.glob("Exercise3_Files/*.mp4") +
    glob.glob("Exercise3_Files/*.mov") +
    glob.glob("Exercise3_Files/*.avi") +
    glob.glob("Exercise3_Files/*.mkv")
)

if not video_files:
    print("No files found")
else:
    print(f"PROCESSING {len(video_files)} FILES:\n")

    Path("outputs/converted").mkdir(parents=True, exist_ok=True)

    invalid_files = []
    report_lines  = []
    report_lines.append("NARBONNE FILM FESTIVAL — FORMAT VALIDATION REPORT\n")
    report_lines.append("=" * 60 + "\n\n")

    # Validate each file
    for filepath in sorted(video_files):
        filename = Path(filepath).name
        print(f"Checking: {filename}")

        is_valid, issues = validate_file(filepath)
```

```
    if is_valid:
        print("VALID\n")
        report_lines.append(f"File: {filename}\n")
        report_lines.append("Compliant : YES\n")
        report_lines.append("Problematic fields: None\n\n")
    else:
        print(f"INVALID ({len(issues)} issues)")
        for issue in issues:
            print(f"~ {issue}")
        print()
        invalid_files.append(filepath)
        report_lines.append(f"File: {filename}\n")
        report_lines.append("Compliant: NO\n")
        report_lines.append("Problematic fields:\n")
        for issue in issues:
            report_lines.append(f"- {issue}\n")
        converted_name = f"{Path(filepath).stem}_formatOK.mp4"
        report_lines.append(f"Converted: {converted_name}\n\n")

# Save txt report
report_path = Path("outputs") / "format_audit_report.txt"
with open(report_path, "w", encoding="utf-8") as f:
    f.writelines(report_lines)
print(f"Report saved: {report_path}")

# Convert all invalid files
```

```
    print(f"\nCONVERTING {len(invalid_files)} FILES:\n")
    success_count = 0
    for filepath in invalid_files:
        output_path = f"outputs/converted/{Path(filepath).stem}_formatOK.mp4"
        print(f"Converting: {Path(filepath).name}")
        success = convert_file(filepath, output_path)
        if success:
            print(f"Saved: {Path(output_path).name}\n")
            success_count += 1
        else:
            print(f"Failed!\n")

    print(f"{success_count}/{len(invalid_files)} converted successfully!")
```

#---Verifying Converted Files---

#~~~Own Implementation – Post Conversion Revalidation Step~~~

```
print("VERIFYING CONVERTED FILES\n")

format_ok_files = sorted(glob.glob("outputs/converted/*_formatOK.mp4"))
perfect_count   = 0

for filepath in format_ok_files:
    print(f"{Path(filepath).name}")

    is_valid, issues = validate_file(filepath)
```

```
data     = probe_file(filepath)
v_stream = next((s for s in data["streams"] if s["codec_type"] == "video"), None)
a_stream = next((s for s in data["streams"] if s["codec_type"] == "audio"), None)
```

#~~~Own Implementation – Detailed Metadata Display~~~

```
if v_stream:
    vbr = int(v_stream.get("bit_rate") or 0)
    print(f"Container    : {data['format'].get('format_name')}")
    print(f"Video codec  : {v_stream['codec_name']}")
    print(f"Resolution   : {v_stream['width']}x{v_stream['height']}")
    print(f"FPS          : {parse_fps(v_stream['r_frame_rate']):.2f}")
    print(f"Video bitrate: {bitrate_to_mbps(vbr)}")
if a_stream:
    abr = int(a_stream.get("bit_rate") or 0)
    print(f"Audio codec  : {a_stream['codec_name']}")
    print(f"Channels     : {a_stream.get('channels')}")
    print(f"Audio bitrate: {bitrate_to_kbps(abr)}")

if is_valid:
    print("100% FESTIVAL COMPLIANT!")
    perfect_count += 1
else:
    print("Issues remaining:")
    for issue in issues:
        print(f"~ {issue}")
```

```
    print("-" * 55)


# Final summary
report_exists = Path("outputs/format_audit_report.txt").exists()
print(f"\nRESULT : {perfect_count}/{len(format_ok_files)} PERFECT")
print(f"Report : {'EXISTS' if report_exists else 'MISSING'}")
print(f"Files  : {len(format_ok_files)} converted")
```

----------End of Task 3----------